

What This Cell Does (Overall)

This code checks:

Where your program is running

What files are present

Whether your PDF folder exists

Basically:

"Am I in the correct folder? Are my PDFs available?"

Step-by-Step Explanation

1. Get Current Working Directory

```
cwd = os.getcwd()
print(f" Current Working Directory:\n{cwd}")
```

 This tells:

Which folder your notebook/code is currently running in

Example output:

C:\Users\hp\...\rag_2_document

2. List Files in Current Folder

```
for item in os.listdir(cwd):
    print(f" - {item}")
```

 This shows:

All files/folders inside current directory

Example:

- data_docum
- notebook.ipynb
- .env

3. Create Path for Data Folder

```
data_path = os.path.join(cwd, "data_docum")
```

👉 This builds:

Full path to your PDF folder

Example:

C:\Users\hp\...\rag_2_document\data_docum

● 4. Check If Folder Exists

```
if os.path.isdir(data_path):
```

👉 This checks:

Does "data_docum" folder exist or not?

● 5. If Folder Exists

```
files = os.listdir(data_path)
```

👉 This lists:

All PDF files inside data_docum

Example:

- combustion_process.pdf
- engine_system.pdf

● 6. If Folder is Empty

```
print("data folder is empty.")
```

👉 Means:

Folder exists but no files inside

● 7. If Folder NOT Found

```
print("data folder NOT found.")
```

👉 Means:

Your path is wrong OR folder is missing

Why This Cell is IMPORTANT

Without this:

PDF reading will fail ❌

FileNotFoundException ❌

👉 This cell helps you debug:

✓ Correct folder path

✓ Files present

✓ Setup is correct

Simple Analogy

Before cooking:

👉 Check kitchen

👉 Check ingredients

This code = checking your "ingredients" (PDFs)

One-Line Summary

This code verifies the working directory and ensures that the data_docum folder and required PDF files are present before processing them.

What You Should See

If everything is correct:

✓ Current directory shown

✓ data_docum folder found

✓ PDF files listed

Next cell

What This Cell Does (Overall)

PDF files → Read → Extract text → Store in dictionary

👉 Output:

```
documents = {  
    "file1.pdf": "full text...",  
    "file2.pdf": "full text..."  
}
```

Step-by-Step Explanation

1. PDF File List

```
pdf_files = [  
    "data_docum/combustion_process_60_pages.pdf",  
    ...  
]
```

👉 This is:

List of all PDF file paths

👉 You are telling Python:

"These are the files I want to read"

2. Function Definition

```
def extract_text_from_pdfs(pdf_paths):
```

👉 You are creating a **function**

Input → list of PDFs

Output → extracted text

3. Create Storage

```
all_texts = {}
```

👉 This is a **dictionary**

Key = file name

Value = full text of that file

Example:

```
{  
  "engine.pdf": "Engine consists of..."  
}
```

● 4. Loop Through Each PDF

```
for path in pdf_paths:
```

👉 This means:

Process each file one by one

● 5. Print File Name

```
print(f"\n Processing: {path}")
```

👉 Just for debugging:

Shows which file is being processed

● 6. Check File Exists

```
if not os.path.exists(path):
```

👉 Prevents error:

If file missing → skip it

● 7. Read PDF

```
reader = PdfReader(path)
```

👉 This loads the PDF

● 8. Initialize Empty Text

```
text = ""
```

👉 This will store:

All pages text combined

● 9. Loop Through Pages

```
for i, page in enumerate(reader.pages):
```

👉 Means:

Read page 1 → page 2 → page 3 ...

● 10. Extract Text

```
page_text = page.extract_text()
```

👉 Converts:

PDF page → plain text

● 11. Append Text

```
if page_text:  
    text += page_text + "\n"
```

👉 Combine all pages:

Page1 + Page2 + Page3 → full document text

● 12. Store in Dictionary

```
all_texts[os.path.basename(path)] = text
```

👉 Example:

```
{  
    "engine_system.pdf": "full extracted text"  
}
```

👉 `basename()` removes folder path

● 13. Print Success

```
print(f" Loaded: {file} | Pages: {len(reader.pages)}")
```

👉 Shows:

File loaded + number of pages

🔴 14. Error Handling

except Exception as e:

👉 If something fails:

Print error instead of crashing

🟢 15. Return Output

return all_texts

👉 Final output:

Dictionary of all documents

🟢 16. Call Function

documents = extract_text_from_pdfs(pdf_files)

👉 Runs the whole process

🟢 17. Print Output

print(documents.keys())

👉 Shows:

Which files were loaded

🧠 Final Data Structure

```
documents = {  
    "combustion.pdf": "text...",  
    "engine.pdf": "text...",  
    "fuel.pdf": "text...",  
    "testing.pdf": "text..."  
}
```

🔥 Why This Cell is VERY IMPORTANT

Without this:

No text ❌

No triples ❌

No graph ❌

No RAG ❌

👉 This is the **starting point of your pipeline**

Simple Analogy

PDF = Book

This code = Reading all books and storing notes

One-Line Summary

This code reads multiple PDF files, extracts their text page by page, and stores the content in a dictionary for further processing in the Graph RAG pipeline.